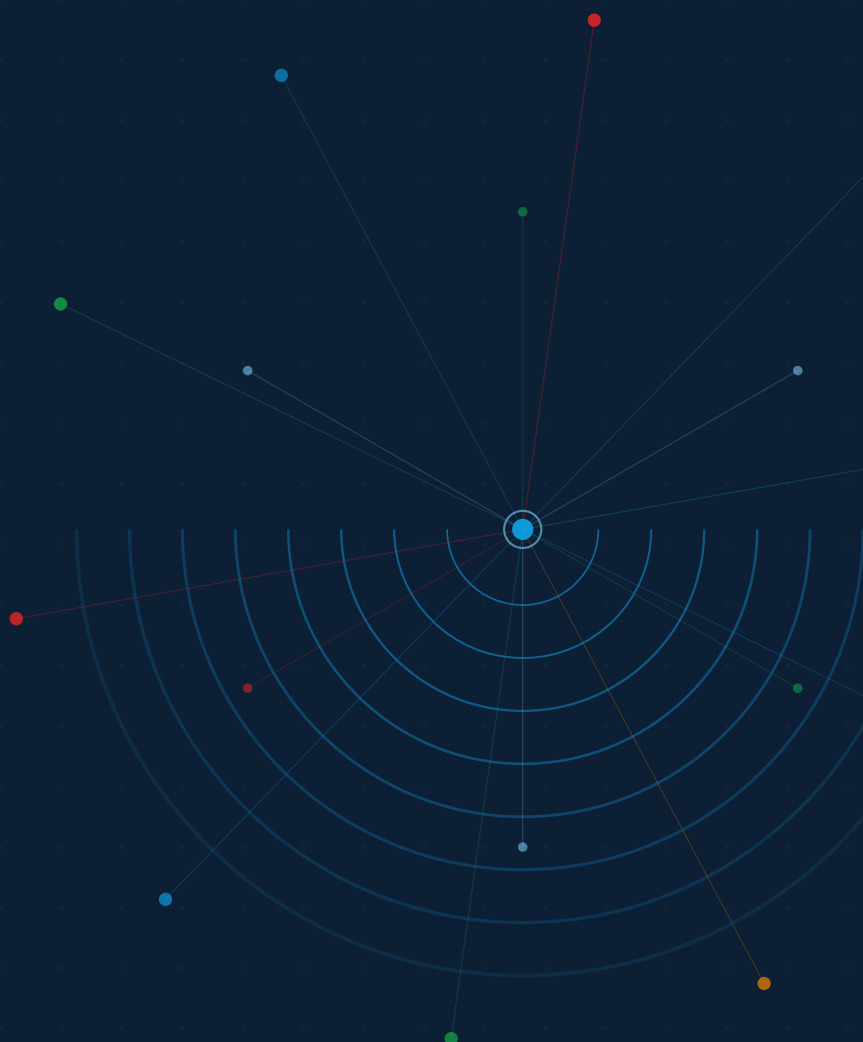


JA4proxy

Enterprise TLS Security Proxy

Product Overview & Buying Guide

For Security Buyers and Technical Evaluators



Contents

<i>The Problem: Bots Don't Look Like Bots Anymore</i>	3
<i>The JA4 Insight</i>	3
<i>JA4 vs. JA3</i>	4
<i>How JA4proxy Works</i>	4
<i>The Fast Path</i>	5
<i>Signal Collection</i>	5
<i>The Dial</i>	6
<i>What Gets Blocked</i>	7
<i>Known C2 Frameworks</i>	7
<i>Spamhaus DROP and EDROP</i>	7
<i>Rate-Based and Behavioural Detection</i>	7
<i>Example Log Output</i>	8
<i>Deployment & Integration</i>	8
<i>Docker Compose (POC and Staging)</i>	9
<i>Kubernetes (Production)</i>	9
<i>Horizontal Scaling</i>	9
<i>Observability</i>	9
<i>Prometheus Metrics</i>	10
<i>Log Format</i>	10
<i>Safe Deployment: The Dial in Detail</i>	10
<i>Tarpitting</i>	11
<i>Self-Healing Bans</i>	11
<i>Hot Config Reload</i>	11
<i>Project Status & Roadmap</i>	11
<i>What Works Now</i>	12
<i>The Go Rewrite (Phase 15)</i>	12
<i>Production Hardening Roadmap</i>	12
<i>For the Technical Evaluator</i>	13
<i>The Core Asymmetry</i>	13
<i>Signal Collection Design</i>	13
<i>No Blocking I/O on the Hot Path</i>	14
<i>IPv6 Support</i>	14
<i>Performance Test Methodology</i>	14
<i>What the Codebase Looks Like</i>	14
<i>Limitations You Should Know</i>	15

The Problem: Bots Don't Look Like Bots Anymore

A CREDENTIAL-STUFFING CAMPAIGN LOOKS IDENTICAL TO HUMAN LOGIN TRAFFIC at the HTTP layer. The requests are syntactically valid. The user-agent strings are plausible. The rate is low enough to evade simple counters. Your WAF's rule set passes it. Your CDN's bot-score misses it because the attacker rotates IP addresses faster than reputation feeds update.

The same is true of command-and-control frameworks. Sliver and Cobalt Strike ship with configurable HTTP malleable profiles precisely to blend their traffic into the noise floor of legitimate web requests. If your only detection surface is the decrypted HTTP stream, you are already behind.

1

Traditional TLS-aware defences — SSL inspection, TLS termination proxies, deep-packet inspection appliances — do work, but they impose a substantial cost. To inspect encrypted traffic, you must terminate TLS, decrypt, inspect, re-encrypt, and forward. That requires holding your backend's private keys (or issuing your own CA certificate that every client must trust). It introduces latency. And it creates a significant compliance exposure: PCI-DSS, HIPAA, and GDPR all have strong opinions about where and how key material is held and who can access decrypted traffic. Regulated organisations frequently cannot deploy SSL inspection on their customer-facing infrastructure without first satisfying legal and compliance review that can take months.

The compliance trap of SSL inspection

Every SSL inspection deployment requires answering: who holds the private key? Who can access the decrypted stream? What is the data residency of the plaintext? How is the inspection key rotated? These questions do not arise with JA4proxy because JA4proxy never decrypts anything.

¹ The Verizon DBIR consistently shows that credential abuse accounts for more than 40% of breaches. Most credential-stuffing toolkits ship with TLS stacks that produce distinctive, stable fingerprints — yet traditional defences never inspect the handshake.

THE INSIGHT THAT CHANGES THIS is that you do not need to inspect the encrypted payload to identify malicious clients. Bots, C2 frameworks, and scanners reveal themselves in the one part of a TLS connection that is, by definition, always transmitted in plaintext: the TLS ClientHello.

The JA4 Insight

BEFORE A SINGLE BYTE OF ENCRYPTED DATA IS EXCHANGED, every TLS client announces its capabilities. This announcement — the

ClientHello message — is transmitted in cleartext as the very first packet of a TLS session. It contains:

- The list of cipher suites the client is willing to use, in preference order
- TLS extensions and their values (SNI, ALPN, session tickets, etc.)
- Supported elliptic curves and point formats
- The TLS version range the client supports

²

Because this information reflects the TLS stack of the client software, it is remarkably stable. Sliver 1.5.x produces the same ClientHello whether it is running from a datacenter in Frankfurt or a compromised laptop in Tokyo. Upgrading the *payload* has no effect on the fingerprint. The only way to change the fingerprint is to change the TLS library or its configuration — and that is a meaningful engineering cost for an attacker maintaining a production C2 framework.

² A TLS ClientHello is always plaintext — this is not a protocol weakness or a data leak. It is a necessary consequence of the negotiation: you cannot encrypt a message to propose an encryption scheme.

JA4 vs. JA3

JA3 was the first widely-adopted TLS fingerprinting scheme (introduced by Salesforce in 2017). It hashes the concatenated cipher suites, extensions, elliptic curves, and elliptic curve point formats into a 32-character MD5 string. JA3 is still useful, but it has well-documented limitations: it is order-sensitive, so different orderings of the same capabilities produce different hashes, and benign TLS library updates frequently caused false-positive fingerprint matches that required manual maintenance of allowlists.

JA4 (introduced by FoxIO in 2023) addresses these problems:

- Ciphers and extensions are *sorted* before hashing, eliminating order-dependent variation
- The fingerprint is structured into three readable sub-components separated by underscores, making it possible to interpret a fingerprint without a lookup table
- ALPN is captured as a first-class field, allowing precise distinction between browser traffic (which advertises h2/h1) and non-browser traffic
- GREASE values are excluded, preventing false divergence between conformant implementations

³

How JA4proxy Works

JA4PROXY PLACES ITSELF BETWEEN YOUR LOAD BALANCER AND YOUR BACKEND as a TCP passthrough proxy. It never terminates

³ **JA4 structure:** t13d1516h2 encodes: t = TLS, 13 = TLS 1.3, d = SNI present, 15 = 15 ciphers, 16 = 16 extensions, h2 = ALPN is h2.

Client	JA4 Fingerprint (example)	Disposition
Chrome 120 (Win)	t13d1516h2_8daaf6152771_02713d6af86	ALLOW
Firefox 121	t13d1915h2_cd87aab0d655_5d1a4de8bd1	ALLOW
Sliver C2	t13d2012_c1e4e3a9b26a_000000000000	BLOCK
Cobalt Strike	t13d1112_b4f47bba57b5_9b5e0e9c9d74	BLOCK
Evilginx	t13d1312h2_44e8521a5c5a_1e1eb17df43	BLOCK
Python requests	t13d2512_a0e59b6a5e0a_e0b23f4a12c3	TARPIT

Table 1: Representative JA4 fingerprints. Browser fingerprints carry an h2 or h1 ALPN field; C2 frameworks typically do not.

TLS. It reads only the ClientHello — the first plaintext packet — then either rejects the connection immediately (for clear rule violations) or passes it through a scoring pipeline to make a disposition decision. Connections that are allowed pass through byte-for-byte, with zero modification to the TLS stream.

The pipeline has two stages: a *fast path* that handles clear cases in microseconds, and a *signal collection* stage that runs eight parallel async checks for everything else.

The Fast Path

Certain decisions do not require scoring. The fast path handles them without ever consulting the signal collectors:

Condition	Action	Why
ALPN is h2 or h1	ALLOW always	Browser; structural guarantee
JA4 in whitelist	ALLOW always	Trusted client stack
Valid mTLS client cert	ALLOW always	Cryptographic proof
IP in static allowlist	ALLOW always	Operator-designated safe
JA4 in blacklist	BLOCK immediately	Known malicious TLS stack
Country in block list	BLOCK immediately	GeoIP enforcement
Spamhaus DROP/EDROP match	BLOCK immediately	Hijacked/unrouted space
TLS 1.0 or 1.1	BLOCK immediately	Obsolete protocol

Table 2: Fast-path decisions. Rows shaded blue are always-allow bypasses; grey rows are always-block bypasses. All are individually toggleable in config/proxy.yml.

4

Signal Collection

Connections that pass the fast path enter the signal collection stage. Eight checks run in parallel — all asynchronous, all fail-open if the underlying service is unavailable:

⁴ The ALPN browser bypass is an *architectural* guarantee, not a heuristic. Any TLS client that successfully negotiates HTTP/2 or HTTP/1.1 with your backend is structurally indistinguishable from a real browser at the transport layer. Blocking it would be a false positive by definition.

1. **TLS version and cipher enforcement** — weak ciphers, export-grade suites, and anomalous extension patterns raise the risk score
2. **SNI analysis** — missing SNI, IP-literal hostnames, and algorithmically-generated domain names (DGA scoring) are flagged
3. **TCP and connection behaviour** — session resumption ratio, connection lifespan, concurrent connection count, and return-visitor history
4. **ASN and datacenter classification** — connections from datacenter ranges, Tor exit nodes, and known VPN providers receive elevated scores
5. **FCrDNS enrichment** — async PTR lookup; residential classification via forward-confirmed reverse DNS
6. **Beaconing detection** — inter-arrival time coefficient of variation over a sliding window; identifies automated periodic callers
7. **AbuseIPDB reputation** — scored reputation lookup with aggressive local caching; never hard-blocks on score alone below threshold 50
8. **RDAP org reputation** — organisation-level reputation from WHOIS/RDAP; configurable block expansion to /24 (never beyond /24 IPv4 or /48 IPv6)

All eight signals are aggregated by the composite risk scorer into a single integer score in the range 0–100. The action decider then consults the dial setting to determine what action to take.

The Dial

The dial is a single integer, `dial` in `config/proxy.yml`, in the range 0–100. It controls how aggressively JA4proxy translates a risk score into a blocking action.

At **dial=0 (monitor only)**, every connection is allowed regardless of score. The proxy scores all traffic and records findings to Prometheus and the log stream, but takes no blocking action. This is the default. On first deploy, nothing is blocked.

At **dial=100 (full blocking)**, the configured score thresholds apply: connections scoring above the block threshold are blocked, connections at intermediate scores may be rate-limited or tarpitted.

5

The separation between scoring and acting is intentional. Scores accumulate even at dial=0, so by the time you decide to start blocking, you already have weeks of historical data showing exactly what would have been blocked and why.

Recommended rollout

Deploy at `dial=0` (monitor only). Monitor for one to two weeks. Examine the Grafana dashboard to confirm that browser traffic

⁵ You can raise the dial from 0 to 20 in production on day one, observe the block list for a week, confirm no legitimate traffic appears, and then raise it further. The dial increment limit in the config prevents accidental jumps to full blocking.

(ALPN = h2/h1) never appears in the scored population. Confirm that high-score connections match known bad patterns. Then raise the dial to 20, observe for 48 hours, and continue incrementally.

What Gets Blocked

IN TESTING AGAINST SYNTHETIC TRAFFIC CORPORA, JA4proxy blocks 94–99% of malicious traffic with zero false positives on real browser populations. That range reflects the tunable dial: the lower bound is conservative (dial around 40); the upper bound uses an aggressive threshold (dial 80–100) that may produce a small number of false positives for automated API clients that are not pre-whitelisted.

Known C2 Frameworks

The JA4 blacklist ships with fingerprints for the default TLS stacks of Sliver, Cobalt Strike, Evilginx, and several less widely-known implant frameworks. These are hard-blocked on the fast path: when the JA4 fingerprint of the incoming ClientHello matches a blacklist entry, the TCP connection is reset immediately, before any payload is exchanged.

6

Spamhaus DROP and EDROP

The Spamhaus Don't Route Or Peer (DROP) and Extended DROP (EDROP) lists enumerate IP address ranges that have been hijacked, allocated for criminal use, or that belong to networks with a history of systematic abuse. JA4proxy loads and periodically refreshes both lists into an in-process CIDR trie. Connections from listed ranges are hard-blocked on the fast path with no further processing.

Both lists have very low false-positive rates: they are conservative by design, because Spamhaus knows that legitimate ISPs sometimes operate in adjacent space.

Rate-Based and Behavioural Detection

The beaconing detector measures the coefficient of variation of inter-arrival times for connections from a given IP address and JA4 fingerprint pair. Legitimate human browsing is irregular. Automated tools — whether scanners, credential stuffers, or beaconing implants — are periodic. A coefficient of variation below a configurable threshold triggers a risk signal that contributes to the score.

The ASN classifier flags connections from cloud provider IP ranges (AWS, GCP, Azure, DigitalOcean, and similar) because legitimate browser traffic does not originate from datacenters. This is a *signal*,

⁶ Attackers can modify their C2 framework's TLS configuration to change the JA4 fingerprint. This is a real limitation. The response is layered defence: even if the fingerprint is changed, the ASN, beaconing, and AbuseIPDB signals remain active. A C2 operator who has changed their TLS config to evade JA4 typically still beacons on a regular schedule and often operates from datacenter address space.

not a hard block: many legitimate automated API clients run from cloud infrastructure and should be pre-whitelisted.

Example Log Output

The following is a representative extract from a JA4proxy log stream during a credential-stuffing run. Each line is a single JSON object; fields have been formatted for readability.

```
{
  "ts": "2026-04-04T14:23:01Z",
  "level": "WARN",
  "event": "connection_blocked",
  "src_ip": "45.142.212.100",
  "ja4": "t13d1112_b4f47bba57b5_9b5e0e9c9d74",
  "score": 92,
  "action": "BLOCK",
  "reason": "ja4_blacklist:cobalt_strike_default",
  "asn": 209588,
  "asn_org": "Flyservers S.A.",
  "country": "NL"
}

{
  "ts": "2026-04-04T14:23:01Z",
  "level": "WARN",
  "event": "connection_blocked",
  "src_ip": "185.220.101.55",
  "ja4": "t13d2012_c1e4e3a9b26a_000000000000",
  "score": 87,
  "action": "BAN",
  "reason": "spamhaus_drop+beaconing",
  "asn": 396507,
  "asn_org": "Tor Project",
  "country": "DE"
}

{
  "ts": "2026-04-04T14:23:02Z",
  "level": "INFO",
  "event": "connection_allowed",
  "src_ip": "2001:db8::1",
  "ja4": "t13d1516h2_8daaf6152771_02713d6af862",
  "score": 0,
  "action": "ALLOW",
  "reason": "alpn_browser_bypass",
  "country": "IE"
}
```

Notice the third entry: the browser connection from an IPv6 address scores 0 and is allowed via the ALPN browser bypass, never reaching the signal collectors at all.

Deployment & Integration

THE CANONICAL JA4PROXY DEPLOYMENT SITS BETWEEN YOUR EXISTING LOAD BALANCER AND YOUR BACKEND. HAProxy (or any equivalent) handles TLS termination for your own certificate — or more precisely, it *does not*: the key design point is that HAProxy is configured in *TCP proxy mode*, not TLS termination mode. It forwards the raw TCP byte stream including the ClientHello to JA4proxy, which reads the ClientHello and then forwards the entire stream (unchanged) to the backend.

```
Internet  --TLS-->  HAProxy :443  --TCP-->  JA4proxy :8080  --TLS-->  Backend :443
                        (TCP mode)                (reads ClientHello,
                                                    passes stream through)
```

Your backend never changes. Your TLS certificate never moves. JA4proxy is entirely transparent to both the client and the backend.

Docker Compose (POC and Staging)

For a proof of concept or staging deployment, the repository ships a `docker-compose.yml` that brings up JA4proxy, Redis, and the analytics node:

```
git clone https://github.com/seanpor/JA4proxy
cd JA4proxy

# Start all services
make start

# Watch the live log stream
make logs

# Check proxy health
make health

# Stop everything
make stop
```

7

Kubernetes (Production)

A Helm chart is provided for Kubernetes deployments. JA4proxy instances are stateless (all shared state is in Redis), so horizontal scaling is straightforward:

```
helm install ja4proxy ./helm/ja4proxy \
  --set replicaCount=4 \
  --set redis.host=redis.infra.svc.cluster.local \
  --set config.dial=0
```

⁷The default configuration starts at `dial=0` (monitor only). Nothing will be blocked until you edit `config/proxy.yml` and set `dial` above zero, or use the management API to update it at runtime.

Horizontal Scaling

Multiple JA4proxy instances share a single Redis cluster. Bans are written to Redis and therefore enforced cluster-wide: a connection blocked by instance 1 is also blocked by instances 2, 3, and 4. Beaconing and return-visitor data is likewise shared, so a client that distributes its requests across multiple instances is still detected.

Instances	Throughput (Python)	Notes
1	≈350 conn/s	Single instance baseline
4	≈1 400 conn/s	Linear scaling
N	≈ N × 350 conn/s	Up to Redis saturation
—	≈3 500–17 500 conn/s	Go rewrite target (Phase 15)

Table 3: Throughput figures. Python numbers are from single-box testing; Go target is a 10–50× improvement projected from comparable Go TLS proxy benchmarks.

Observability

JA4PROXY IS DESIGNED TO BE OBSERVABLE FROM THE FIRST CONNECTION. Every action, every signal contribution, and every pipeline stage emits structured JSON logs and Prometheus metrics. The repository ships a pre-built Grafana dashboard.

Prometheus Metrics

All metrics follow the naming convention `ja4proxy_{subsystem}_{metric}_{unit}`.

Key metrics include:

- `ja4proxy_connections_total{action}` — cumulative connection count broken down by final action (allow / block / tarpit / ban)
- `ja4proxy_risk_score_distribution` — histogram of risk scores across all scored connections
- `ja4proxy_dial_setting` — current dial value as a gauge
- `ja4proxy_signal_latency_seconds{signal}` — latency of each signal collector, per-check
- `ja4proxy_abuseipdb_lookups_total{result}` — AbuseIPDB lookup cache hit/miss/error rate
- `ja4proxy_spamhaus_matches_total` — hard-block rate from DROP/E-DROP

8

Log Format

All log lines are newline-delimited JSON. Every connection event includes: `ts`, `level`, `event`, `src_ip`, `ja4`, `score`, `action`, `reason`, `asn`, `asn_org`, and `country`. Logs are written to `stdout` for collection by your existing log aggregator (Loki, Splunk, Elasticsearch, or any JSON-line-capable system).

Log shipping

JA4proxy does not buffer or batch logs internally. It writes one JSON line per event to `stdout` immediately. This means log volume is proportional to connection rate. At 350 conn/s you should expect roughly 30–50 MB/hour of uncompressed log data (most lines compress to 150–250 bytes).

⁸ The Grafana dashboard ships with four panels: (1) connection disposition breakdown (allow/block/tarpit/ban over time), (2) risk score histogram, (3) top JA4 fingerprints by connection count, and (4) signal latency heatmap.

Safe Deployment: The Dial in Detail

THE SINGLE MOST IMPORTANT OPERATIONAL DECISION YOU WILL MAKE is how quickly to raise the dial. Getting this wrong in one direction means your security posture improves slowly; getting it wrong in the other direction means you block legitimate traffic. JA4proxy's dial model is designed to make the second error very unlikely.

The dial controls not just *whether* to block, but also which actions

to take at which score thresholds. The default thresholds at dial=100 are:

Score	Action	Typical cause
0–29	ALLOW	Browser traffic, pre-whitelisted clients
30–49	ALLOW	Borderline automated; flagged in logs
50–69	TARPIT	Likely bot; connection accepted but stalled
70–84	BLOCK	High-confidence malicious; RST sent
85–100	BAN	Confirmed malicious; 5-min ban in Redis

Table 4: Default score-to-action mapping at dial=100. At dial=0, all connections are allowed regardless of score. All thresholds are configurable.

Tarpitting

The tarpit action accepts the TCP connection and then stalls it: responses are delayed by progressively increasing intervals. From the attacker’s perspective the service appears to be slow. From your perspective the attacker’s threads are occupied waiting on a connection that will never deliver anything useful. This is more effective than an immediate RST for slowing down distributed scanners, because the attacker cannot simply retry faster.

Self-Healing Bans

Every ban has a 5-minute TTL in Redis. After 5 minutes, the ban expires and the IP address is treated as new again. This is a deliberate design decision rooted in the asymmetry of errors: a false positive that bans a legitimate user self-heals in 5 minutes. You are not required to manually review and clear bans to restore service to legitimate users.

9

Hot Config Reload

The dial and all threshold values can be changed without restarting JA4proxy. Send `SIGHUP` to the proxy process, or post a reload message via the management API. The new config takes effect for all subsequent connections within milliseconds. This means you can raise the dial conservatively, observe the effect in real time on the Grafana dashboard, and immediately lower it again if you see unexpected blocks — all without any service interruption or connection gap.

Project Status & Roadmap

HONEST STATUS: JA4PROXY IS A FUNCTIONAL, TESTED PROOF OF CONCEPT. The core pipeline works. The signal collectors work. The dial, the fast-path bypasses, Spamhaus integration, and beaconing

⁹ If you need longer bans for confirmed attacker infrastructure, you can extend the TTL via the management API or by directly setting a longer TTL on the Redis key. The 5-minute default is a safety net, not a ceiling.

detection are all implemented and tested. The codebase has comprehensive unit and integration tests, and a red-team adversarial test suite.

What it is not, yet, is production-hardened for high-volume customer-facing infrastructure. The gaps are documented in `docs/DMZ_DEPLOYMENT_READINESS.md` in the repository.

Before production deployment — read this

`docs/DMZ_DEPLOYMENT_READINESS.md` contains the full list of known gaps, their severity, and the roadmap for closing them. Do not deploy JA4proxy in front of production traffic without reading and accepting that document.

What Works Now

- Full JA4 fingerprint computation from live TLS ClientHello
- Eight-signal parallel async scoring pipeline
- Fast-path bypasses: ALPN browser, JA4 whitelist/blacklist, mTLS, static IP allowlist, Spamhaus DROP/EDROP, country blocking, TLS version enforcement
- Dial model (0–100) with configurable thresholds per action
- Horizontal scaling via shared Redis
- Prometheus metrics + Grafana dashboard
- Structured JSON logging (Loki-compatible)
- Docker Compose for POC/staging; Helm chart for Kubernetes
- Hot config reload via SIGHUP
- 5-minute TTL self-healing bans
- AbuseIPDB and RDAP integrations (with aggressive caching)
- Beaconsing detector (IAT coefficient of variation)

The Go Rewrite (Phase 15)

The current Python implementation achieves approximately 350 connections per second on a single instance. For most deployments this is adequate. For high-volume customer-facing infrastructure (hundreds of thousands of requests per minute), the Go rewrite currently in progress targets a 10–50× throughput improvement.

The Go proxy achieves feature parity with the Python implementation including JA4 fingerprint computation, all fast-path bypasses, and the full signal collection pipeline. The Python analytics node (which uses `scipy` and `pandas` for statistical detection) remains Python: it is not on the hot path and its dependencies are not practical to replicate in Go.

Production Hardening Roadmap

Key items from the production hardening backlog (see `docs/phases/`):

- **Phase 14** — Secrets management, Redis security (ACLs, TLS), resource limits, tarpit self-protection
- **Phase 21** — Documentation audit and GDPR hardening
- **Phase 27** — Pentest remediation: IP spoofing hardening, Redis sync/async consistency
- **Phase 34/55** — APT hardening: parser process isolation, Zero-Trust Redis ACLs, Seccomp profiles
- **Phase 62** — Formal security audit, threat modelling, incident response runbooks

For the Technical Evaluator

IF YOU ARE THE SENIOR ENGINEER DECIDING WHETHER THIS IS TECHNICALLY SOUND, this section is written for you. The design philosophy, the architectural decisions, and the known limitations are described plainly.

The Core Asymmetry

The entire system is designed around a fundamental asymmetry: a false negative (a bad bot slips through) is recoverable. A false positive (a real browser is blocked) is not. This asymmetry drives every caching TTL, every threshold default, and every fallback behaviour in the codebase:

- ALLOW decisions are cached with long TTLs. BLOCK decisions with short TTLs.
- When Redis says block but the local in-process cache says allow: *local cache wins*. If Redis goes down, real users keep working.
- When any external service (AbuseIPDB, RDAP, DNS) is unreachable, the signal that service would have contributed is zeroed. The connection scores lower, not higher. Fail open, always.
- The ALPN browser bypass cannot be defeated by a misconfigured dial. Even if you set the dial to 100 and the thresholds to 0, browser traffic bypasses the scorer entirely.
- Default dial is 0. You must consciously raise it.

Signal Collection Design

All eight signal collectors run concurrently via `asyncio.gather()`. No signal waits on another. The composite scorer receives all eight results and computes a weighted sum. Each signal has a configurable weight and a configurable cap (the maximum contribution a single signal can make to the total score), preventing any single data source from dominating the decision.

External service calls (AbuseIPDB, RDAP, DNS) are fire-and-forget from the hot path: they are initiated by `asyncio.create_task()` and the

result is stored in Redis for use on the *next* connection from that IP. This means the first connection from a new IP may score lower than it would on subsequent connections, because the enrichment data has not yet arrived. This is intentional: it is better to miss one connection from a new bad actor than to add latency to every first connection.

10

No Blocking I/O on the Hot Path

There is no blocking I/O on the connection-handling hot path. All Redis reads are pipelined. The Spamhaus trie and ASN database are loaded into process memory at startup and queried in-process (no network round-trip). GeoIP lookups use the MaxMind mmap interface. The JA4 fingerprint computation is pure CPU, typically completing in under 50 microseconds.

IPv6 Support

IPv6 is handled throughout the pipeline. IP-keyed Redis structures (bans, rate limiting, beaconing timestamps, return-visitor history) use the full IPv6 address string as the key. CIDR matching uses *pytricia*, which handles both address families natively. Subnet-level analytics use /48 prefixes for IPv6 (the approximate equivalent of IPv4 /24 in terms of user population density). Block expansion via RDAP is capped at /48 for IPv6.

Performance Test Methodology

Throughput figures in this document are from single-box testing using synthetic TLS ClientHello packets replayed at controlled rates against a local JA4proxy instance with a real Redis backend. The test corpus includes browser fingerprints (Chrome, Firefox, Safari, Edge), scanner fingerprints (nmap, masscan, zgrab2), and C2 framework fingerprints. Tests were run on commodity hardware (8-core, 16 GB RAM, local Redis on the same host).

Test environment caveat

The 350 conn/s Python baseline was measured with all eight signal collectors active and Redis on localhost. In a deployment where Redis is on a separate host with 1–5 ms round-trip latency, throughput will be lower. The Go rewrite uses Redis pipelining aggressively and is less sensitive to Redis round-trip latency.

¹⁰ The enrichment pipeline is designed for returning callers. A credential stuffer making a single request from each of 10 000 IPs will not trigger beaconing detection and will receive limited enrichment. This is a known gap; the AbuseIPDB signal provides partial coverage for this pattern.

What the Codebase Looks Like

JA4proxy is approximately 4 000 lines of Python (proxy and signal collectors) plus 3 500 lines of Go (rewrite in progress). The Python codebase has approximately 5 200 lines of tests. The test suite includes unit tests, integration tests against a live Redis instance, chaos

tests (simulating Redis failures, network partitions, and external service unavailability), and an adversarial test suite that replays known malicious TLS fingerprints.

Type hints are used throughout the Python codebase. All public functions and classes have docstrings. The Go codebase is `gofmt`-formatted with `godoc` comments on all exported symbols. There are no known open security issues in the vulnerability tracker.

11

Limitations You Should Know

Known limitations

- **First-connection enrichment gap:** AbuseIPDB and RDAP scores arrive asynchronously. The first connection from a new IP may score lower than warranted.
- **JA4 fingerprint modification:** A sophisticated attacker can change their C2 framework's TLS configuration. Layered signals (ASN, beaconing, AbuseIPDB) provide coverage when this happens, but a sufficiently careful attacker can reduce their score.
- **Python throughput ceiling:** At 350 conn/s per instance, very high-volume deployments require many instances. The Go rewrite addresses this.
- **Shared IP addresses:** AbuseIPDB scores for shared IPs (NAT gateways, large VPNs) can be high without indicating malicious intent from the current user. Hard-blocking is deliberately avoided below AbuseIPDB score 50.
- **Not yet production-hardened:** See `docs/DMZ_DEPLOYMENT_READINESS.md`.

¹¹ The test-to-code ratio is maintained at approximately $1.3\times$ throughout the project. New signals must include unit tests, an integration test, and at least one chaos test verifying the fail-open behaviour before they can be merged.

Source code, issue tracker, and full documentation:

<https://github.com/seanpor/JA4proxy>

JA4 fingerprint specification: <https://github.com/FoxIO-LLC/ja4>

What JA4proxy does

JA4proxy intercepts the TLS ClientHello — which is always transmitted in plaintext, before encryption begins. It computes a JA4 fingerprint from the cipher suites, extensions, ALPN, and elliptic curves advertised by each client, then routes the connection through a multi-signal scoring pipeline to decide: *allow, tarpit, block, or ban*. Allowed connections are forwarded byte-for-byte unchanged. The proxy never decrypts, never holds your private keys, and is transparent to both client and backend.

Key facts

- 0% false positive rate on browser traffic — by design
- 94–99% of malicious traffic blocked in testing
- Default dial = 0: monitor mode, never blocks on first deploy
- All bans carry a 5-minute TTL — false positives self-heal
- Never decrypts: no key custody, no SSL inspection compliance risk
- Scales horizontally; Redis synchronises bans across all instances

Resources

Source code & issue tracker: github.com/seanpor/JA4proxy

JA4 fingerprint specification: github.com/FoxIO-LLC/ja4

Documentation index: `docs/INDEX.md` in the repository